

AA-01: A proposed decision procedure for accurate polynomial evaluation

Exact real inputs, independently rounded arithmetic,
exact comparisons, and no constant inputs

George Stepaniants

Department of Computing and Mathematical Sciences
California Institute of Technology
Pasadena, California, USA

11 September 2026

Abstract

We give a proof candidate for an affirmative answer to AA-01 in its constant-free, finite-computation-tree formulation. The proposed characterization uses the finitely many regions determined by input signs and the ordering of input absolute values. After changing to nonnegative gap coordinates on each region, accurate evaluability is equivalent to a uniform bound of the coefficientwise absolute polynomial by the absolute value of the polynomial itself. Each bound is a first-order sentence over the reals, so effective real quantifier elimination gives a halting decision procedure. The necessity argument couples two nearby executions while preserving error-dependent branches and all stored rounded values. The sufficiency argument constructs an evaluator using only exact sign/order branching and rounded evaluation in the gap coordinates.

Verification status. This is a research proof candidate developed in the accompanying conversation with ChatGPT. A separate Codex agent independently reviewed the proof on 11 September 2026. This review is not human peer review or a formal verification certificate. All model-specific lemmas are proved below; the only external algorithmic theorem used is effective quantifier elimination over the reals. The references supply the problem, background, and that external theorem, not a prior verification of the proposed characterization.

1 The precise problem and computation model

Fix an integer $n \geq 1$. The input to the decision problem is the finite integer coefficient list of a polynomial

$$p \in \mathbb{Z}[x_1, \dots, x_n], \quad p(0) = 0.$$

The model is the one specified in AA-01 [1]. Background on accurate evaluation in traditional rounded arithmetic is given in [2, 3].

An evaluator is a finite arithmetic computation tree with real-valued registers and branching control flow. Its initial numerical quantities are exactly the input coordinates $x_1, \dots, x_n$. Each binary arithmetic node performs one of $+$, $-$, or multiplication and returns

$$(a \circ b)(1 + \delta_j), \quad \circ \in \{+, -, \cdot\}. \quad (1.1)$$

Every execution of a distinct arithmetic node has its own independent error parameter. Here “independent” imposes no probability distribution: it means that the errors are separately and arbitrarily selectable within the prescribed interval. Equal operands at different nodes need not produce equal errors.

Negation, copying, storage, and comparisons of available numerical quantities are exact. A comparison chooses control flow; its Boolean outcome is not an additional numerical constant available to arithmetic. The output is an available numerical quantity. There are no constant input nodes and no additional exact arithmetic operations. In particular, exact multiplication by an integer is not a primitive. Previously computed values may be reused without being recomputed. The tree is finite, including all its branches. Arithmetic is over the reals, without overflow or underflow.

Number all rounded nodes in the tree by $1, \dots, N(P)$. Errors assigned to nodes on unused branches are ignored. Write $P(x, \delta)$ for the resulting output.

Definition 1.1 (Accuracy). A tree P accurately evaluates p if $P(x, 0) = p(x)$ for all $x \in \mathbb{R}^n$ and

$$\begin{aligned} \forall \eta \in (0, 1) \exists u \in (0, 1) \quad \forall x \in \mathbb{R}^n \forall \delta \in [-u, u]^{N(P)} : \\ |P(x, \delta) - p(x)| \leq \eta |p(x)|. \end{aligned} \quad (1.2)$$

The same tree must work for every η . The precision bound u may depend on p , P , and η , but not on x .

The question is whether an ordinary Turing algorithm always halts and decides whether such a tree exists. The Turing decision procedure and the rounded evaluator are different machines: the former may perform exact symbolic integer computations and invoke a real-algebraic decision algorithm.

2 The proposed finite characterization

For $\pi \in \mathfrak{S}_n$ and $s = (s_1, \dots, s_n) \in \{-1, 1\}^n$, define $T = T_{\pi, s} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ by

$$(Ty)_{\pi(i)} = s_i \sum_{j=i}^n y_j, \quad 1 \leq i \leq n. \quad (2.1)$$

This is an invertible integer linear map. Its inverse on its closed signed-order region is

$$y_i = \left| x_{\pi(i)} \right| - \left| x_{\pi(i+1)} \right| \quad (1 \leq i < n), \quad y_n = \left| x_{\pi(n)} \right|. \quad (2.2)$$

For $y \in (0, \infty)^n$, the image satisfies

$$\text{sgn}(x_{\pi(i)}) = s_i, \quad \left| x_{\pi(1)} \right| > \dots > \left| x_{\pi(n)} \right| > 0.$$

The $2^n n!$ closed regions $T_{\pi, s}[0, \infty)^n$ cover $\mathbb{R}^n$. For zero coordinates choose either sign, and for ties choose any order.

Expand, after collecting like monomials,

$$q_T(y) = p(Ty) = \sum_{\alpha \in \mathbb{N}_0^n} a_{T, \alpha} y^\alpha, \quad W_T(y) = \sum_{\alpha \in \mathbb{N}_0^n} |a_{T, \alpha}| y^\alpha. \quad (2.3)$$

The sums are finite, their coefficients are integers, and their constant terms are zero. We use the polynomial convention $y_i^0 = 1$, including when $y_i = 0$.

Theorem 2.1 (Signed-order gap criterion). *For the model of Section 1, the following are equivalent.*

- (i) *The polynomial p has an accurate finite tree.*
- (ii) *For every $T = T_{\pi,s}$, there is $C_T > 0$ such that*

$$W_T(y) \leq C_T |q_T(y)| \quad \text{for all } y \in [0, \infty)^n. \quad (2.4)$$

When these conditions hold, an accurate evaluator can be constructed whose only comparisons determine the exact signs and the ordering of the absolute values of the original inputs.

Necessity is proved in Sections 3–5; sufficiency is proved in Section 6. The halting decision procedure follows in Section 7. The zero polynomial will be dealt with explicitly and causes no exception to the criterion.

3 Geometry of a signed-order region

Write $S_i(y) = \sum_{j=i}^n y_j$. Up to sign and permutation, the raw inputs are the nested tail sums $S_1(y), \dots, S_n(y)$.

Lemma 3.1 (Relative perturbations of raw quantities). *Fix $T = T_{\pi,s}$, $0 < \varepsilon < 1$, and $y, y' \in (0, \infty)^n$ with*

$$|y'_i - y_i| \leq \varepsilon y_i \quad (1 \leq i \leq n). \quad (3.1)$$

Let $x = Ty$ and $x' = Ty'$. Then:

- (a) *Every signed raw input $r = \sigma x_i$, $\sigma \in \{-1, 1\}$, and its counterpart $r' = \sigma x'_i$ satisfy*

$$|r' - r| \leq \varepsilon |r|. \quad (3.2)$$

- (b) *A sum or difference L of two signed raw inputs is either identically zero on the region, or is nonzero throughout the open region and satisfies*

$$|L(x') - L(x)| \leq \varepsilon |L(x)|. \quad (3.3)$$

In particular, comparisons between signed raw inputs have the same outcomes at x and x' .

- (c) *A product a of two signed raw inputs and its counterpart a' satisfy*

$$|a' - a| \leq (2\varepsilon + \varepsilon^2) |a| \leq 3\varepsilon |a|. \quad (3.4)$$

Proof. For any $b_i \geq 0$, condition (3.1) implies

$$\left| \sum_i b_i y'_i - \sum_i b_i y_i \right| \leq \sum_i b_i |y'_i - y_i| \leq \varepsilon \sum_i b_i y_i.$$

This proves (a). A sum or difference of two signed tail sums is either zero or, up to an overall sign, a nonzero linear form with nonnegative coefficients: $S_i + S_j$ has this property, and, for $i < j$, $S_i - S_j = \sum_{k=i}^{j-1} y_k$ does too. The displayed inequality proves (b), including preservation of signs and exact equality cases.

For (c), write the two perturbed factors as $r(1 + \theta)$ and $t(1 + \phi)$, with $|\theta|, |\phi| \leq \varepsilon$, using (a). Their relative product change is $\theta + \phi + \theta\phi$, whose absolute value is at most $2\varepsilon + \varepsilon^2$. $\square$

4 Coupling executions, including error-dependent branches

The next lemma permits each rounded value to be selected away from the finitely many signed raw inputs.

Lemma 4.1 (Avoiding finitely many relative neighborhoods). *Let $0 < u < 1/4$, $n \geq 1$, and $\tau = u/(32n)$. Let $\mathcal{R}$ be a set of at most $2n$ nonzero real numbers. For every $a \in \mathbb{R}$, there is $\delta \in [-u/4, u/4]$ such that*

$$|a(1 + \delta) - r| > \tau |r| \quad (r \in \mathcal{R}). \quad (4.1)$$

Proof. If $a = 0$, every δ works because $\tau < 1$ and $r \neq 0$. Suppose $a \neq 0$, and put $I = [-u/4, u/4]$, an interval of length $u/2$. For a fixed r , its forbidden set is

$$F_r = \{\delta \in I : |a(1 + \delta) - r| \leq \tau |r|\}.$$

It is an interval intersected with I and has length at most $2\tau |r| / |a|$. If it is nonempty, some $\delta \in I$ gives

$$(1 - \tau) |r| \leq |a(1 + \delta)| \leq (1 + u/4) |a|.$$

Hence

$$\text{length}(F_r) \leq 2\tau \frac{1 + u/4}{1 - \tau} < 3\tau.$$

For the last inequality, $u/4 < 1/16$ and $\tau < 1/128$, so the factor multiplying τ is less than $272/127 < 3$. The union of all forbidden intervals has length at most

$$6n\tau = \frac{3u}{16} < \frac{u}{2}.$$

It cannot cover I . A point of its complement satisfies every strict inequality in (4.1). $\square$

Lemma 4.2 (Deterministic coupling). *Fix a finite tree P in the prescribed model and $0 < u < 1/4$. Set*

$$\tau = \frac{u}{32n}, \quad \varepsilon = \frac{u\tau}{16} = \frac{u^2}{512n}. \quad (4.2)$$

Suppose $x = Ty$, $x' = Ty'$ for one common map T , where $y, y' > 0$ and (3.1) holds. There are error assignments

$$\delta, \delta' \in [-u, u]^{N(P)}$$

with the following properties:

- (i) *the two executions follow exactly the same control-flow path;*
- (ii) *every rounded node on that path produces the same numerical value in the two executions;*
- (iii) *if their outputs are $z = P(x, \delta)$ and $z' = P(x', \delta')$, then*

$$|z'| \leq (1 + \varepsilon) |z|. \quad (4.3)$$

The parameter ε depends only on n and u , not on the size of P .

Proof. Every available numerical quantity has one of two provenance types: it is an exact signed copy of an original input coordinate, or an exact signed copy of a value previously produced by rounded arithmetic. Copies and negations preserve these types. They are provenance labels, not claims that differently labelled values can never coincide.

We construct the two executions simultaneously, assigning an error when a rounded node is reached. For every rounded value v produced in the execution on x , maintain

$$|v - r| > \tau |r| \quad \text{for all } r \in \mathcal{R}_x := \{x_1, -x_1, \dots, x_n, -x_n\}. \quad (4.4)$$

All targets are nonzero because $y > 0$. The same separation holds for $-v$, since $\mathcal{R}_x$ is symmetric under negation. We also maintain that the counterpart of every such rounded value is numerically identical in the second execution. Signed raw inputs instead have the naturally perturbed counterparts from Lemma 3.1.

Consider corresponding arithmetic nodes, with exact results a, a' before the new rounding. The following cases exhaust all their operands.

Both operands have rounded provenance. The operands match exactly, so $a' = a$, including when $a = 0$.

Both operands have raw-input provenance. For addition or subtraction, Lemma 3.1(b) gives either $a = a' = 0$ or a relative change bounded by ε . For multiplication, both factors are nonzero and part (c) bounds the relative change by 3ε .

One operand has each provenance type, with multiplication. Write them as a matched value v and a signed raw input r . If $v = 0$, then $a = a' = 0$. Otherwise, (3.2) bounds the relative change by ε .

One operand has each provenance type, with addition or subtraction. Up to an overall sign, $a = v \pm r$, where v is a signed copy of a prior rounded value. The invariant gives

$$|a| > \tau |r|, \quad |a' - a| \leq \varepsilon |r|.$$

In particular, $a \neq 0$, and

$$\frac{|a' - a|}{|a|} < \frac{\varepsilon}{\tau} = \frac{u}{16}. \quad (4.5)$$

Thus every zero case has $a' = 0$, and every nonzero case has

$$a' = a(1 + \theta), \quad |\theta| \leq u/16. \quad (4.6)$$

Here the raw-input cases use $3\varepsilon \leq u/16$, which follows from (4.2) and $u < 1/4$, $n \geq 1$.

By Lemma 4.1, choose the new error $\delta_j \in [-u/4, u/4]$ so that the first execution's new value $v_{\text{new}} = a(1 + \delta_j)$ satisfies (4.4). If $a = 0$, choose $\delta'_j = 0$ in the second execution; both new values are zero. If $a \neq 0$, set

$$\delta'_j = \frac{1 + \delta_j}{1 + \theta} - 1. \quad (4.7)$$

Then

$$a'(1 + \delta'_j) = a(1 + \delta_j), \quad |\delta'_j| \leq \frac{u/4 + u/16}{1 - u/16} = \frac{5u}{16 - u} < u.$$

The new rounded values match and the invariant is preserved.

It remains to check each comparison, since its operands may depend on rounding errors. Two quantities of rounded provenance match in both executions, so their comparison is identical, including equality. Comparisons between signed raw inputs are preserved by Lemma 3.1. For a mixed comparison, its difference $v - r$ has absolute value greater than $\tau |r|$, whereas the perturbation of r has absolute value at most $\varepsilon |r|$. Since $\varepsilon < \tau$, its strict sign cannot change, and equality cannot occur in either execution. This also covers signed copies of rounded values by symmetry of the target set.

Starting with the input registers, these arguments inductively preserve the path through exact comparisons, negations, copies, and rounded nodes. They do not recompute stored quantities or assign new errors to a reuse. The path is finite. The chosen errors can therefore be extended to full vectors for the two finite trees by assigning zero to every unused node. The independent-error convention makes these legal assignments, even though they were constructed successively.

The common output location either has rounded provenance, in which case $z' = z$, or is a signed raw-input selection, in which case (3.2) gives (4.3). The argument also includes paths with no rounded nodes. $\square$

Remark 4.3. The lemma does not assert that an individual leaf's zero-error expression equals p on all inputs. It compares two actual executions with their selected errors. Nor does it assume that a leaf is reachable at zero error. These distinctions are essential when branching depends on rounded intermediate values.

5 Necessity of the criterion

Proposition 5.1 (Local control under gap perturbations). *If P accurately evaluates p , then there is $\varepsilon > 0$ such that, for every T and all $y, y' > 0$ satisfying (3.1),*

$$|p(Ty')| \leq 2 |p(Ty)|. \quad (5.1)$$

The same ε works for every T and every y .

Proof. Use (1.2) with $\eta = 1/4$. By shrinking its precision bound, take $0 < u < 1/4$. Use the ε in Lemma 4.2. For the coupled outputs z, z' ,

$$|z - p(Ty)| \leq \frac{1}{4} |p(Ty)|, \quad |z' - p(Ty')| \leq \frac{1}{4} |p(Ty')|.$$

Therefore

$$\begin{aligned} |p(Ty')| &\leq \frac{4}{3} |z'| \leq \frac{4}{3} (1 + \varepsilon) |z| \\ &\leq \frac{5}{3} (1 + \varepsilon) |p(Ty)| \leq 2 |p(Ty)|. \end{aligned}$$

The last inequality follows, for example, from $\varepsilon = u^2/(512n) < 1/8192 < 1/5$. No division by $p(Ty)$ is used, so zeros are included. $\square$

We next give an explicit interpolation proof of the finite-dimensional coefficient bound needed below.

Lemma 5.2 (Coefficient extraction on a fixed box). *Fix $n \geq 1$, an integer $d \geq 0$, and $0 < \varepsilon < 1$. There is a finite constant $K = K(n, d, \varepsilon) > 0$ such that every real polynomial $R(t_1, \dots, t_n)$ of total degree at most d satisfies*

$$\|R\|_{\text{coef}} := \sum_{\alpha} |\text{coeff}_{\alpha} R| \leq K \sup_{t \in [1-\varepsilon, 1+\varepsilon]^n} |R(t)|. \quad (5.2)$$

Proof. For $d = 0$, take $K = 1$. For $d \geq 1$, choose $d + 1$ distinct points $t_0, \dots, t_d$ in $[1 - \varepsilon, 1 + \varepsilon]$ and define the univariate Lagrange polynomials

$$L_j(t) = \prod_{\substack{0 \leq k \leq d \\ k \neq j}} \frac{t - t_k}{t_j - t_k}.$$

Successive univariate interpolation in each coordinate gives

$$R(t) = \sum_{j_1=0}^d \cdots \sum_{j_n=0}^d R(t_{j_1}, \dots, t_{j_n}) \prod_{i=1}^n L_{j_i}(t_i).$$

This applies because the degree in each separate variable is at most d . The coefficient norm is subadditive. Because the factors in the product involve distinct variables, the coefficient norm of that product is $\prod_i \|L_{j_i}\|_{\text{coef}}$. Thus (5.2) holds with the finite positive constant

$$K = \left(\sum_{j=0}^d \|L_j\|_{\text{coef}} \right)^n.$$

□

Proof of Theorem 2.1, necessity. Suppose p has an accurate tree. If $p = 0$, then $q_T = W_T = 0$ for every T , and the criterion is immediate. Otherwise let $d = \deg p$ and use ε from Proposition 5.1.

Fix T and $y \in (0, \infty)^n$. For $t \in [1 - \varepsilon, 1 + \varepsilon]^n$, the point $y' = t \odot y = (t_1 y_1, \dots, t_n y_n)$ satisfies (3.1). Consequently,

$$\sup_{t \in [1-\varepsilon, 1+\varepsilon]^n} |q_T(t \odot y)| \leq 2 |q_T(y)|. \quad (5.3)$$

Apply Lemma 5.2 to the polynomial in t

$$R_y(t) = q_T(t \odot y) = \sum_{\alpha} a_{T,\alpha} y^{\alpha} t^{\alpha}.$$

Since $y > 0$, its coefficient norm is exactly $W_T(y)$. Equation (5.3) yields

$$W_T(y) \leq 2K |q_T(y)|.$$

Both sides are continuous functions of y , so the same inequality holds on the closure $[0, \infty)^n$. This proves (2.4), with $C_T = 2K$, for each T . □

6 Constructing an accurate evaluator

For completeness, the following evaluation bound explicitly retains all error factors introduced by stored-value reuse.

Lemma 6.1 (Termwise evaluation with relative input errors). *Let*

$$q(y) = \sum_{\alpha \neq 0} a_{\alpha} y^{\alpha} \in \mathbb{Z}[y_1, \dots, y_n]$$

be nonzero with zero constant term. Suppose the available nonnegative data have the form $\hat{y}_i = y_i(1 + \xi_i)$, with $y_i \geq 0$ and $|\xi_i| \leq u < 1$. There is a fixed constant-free arithmetic program for q , using these data, with output $\hat{q}$ and an integer $L \geq 0$ such that

$$|\hat{q} - q(y)| \leq ((1 + u)^L - 1) \sum_{\alpha \neq 0} |a_{\alpha}| y^{\alpha}. \quad (6.1)$$

Here the same bound u applies to all its new arithmetic errors, and L depends only on the chosen program and q .

Proof. For every nonzero coefficient, form y^α by multiplying its $|\alpha| = \sum_i \alpha_i \geq 1$ factors, beginning with its first factor rather than with a constant 1. To implement its integer coefficient, take $|a_\alpha|$ occurrences of this computed monomial in a sum and use exact negation if $a_\alpha < 0$. Then add all these signed occurrences, starting with the first one rather than a constant 0. This is a finite program; large coefficients affect its size, not its existence. Reuse of the computed monomial is permitted.

Substitute each input factor $y_i(1 + \xi_i)$ into this program and expand only the additions, including their rounding factors. If ζ lists all input-error and arithmetic-error variables, its output has the form

$$\hat{q} = \sum_{r=1}^M \sigma_r y^{\alpha(r)} \prod_j (1 + \zeta_j)^{e_{rj}}, \quad \sigma_r \in \{-1, 1\}, \quad e_{rj} \in \mathbb{N}_0. \quad (6.2)$$

There are exactly $|a_\alpha|$ occurrences with exponent α , and all these occurrences have sign $\text{sgn}(a_\alpha)$. In particular,

$$\sum_{r=1}^M \sigma_r y^{\alpha(r)} = q(y), \quad \sum_{r=1}^M y^{\alpha(r)} = \sum_{\alpha} |a_\alpha| y^\alpha.$$

The exponents e_{rj} retain multiplicities of shared input errors and shared stored values; no independent error is assigned to a reuse. The expansion (6.2) follows directly because no multiplication of two sums is required in the chosen program.

Let $L = \max_r \sum_j e_{rj}$. When $|\zeta_j| \leq u < 1$, each error product lies between $(1 - u)^L$ and $(1 + u)^L$. Furthermore,

$$1 - (1 - u)^L \leq (1 + u)^L - 1,$$

by the binomial theorem (also when $L = 0$). Thus every error product in (6.2) differs from 1 by at most $(1 + u)^L - 1$. Subtract $q(y)$ and use the triangle inequality to obtain (6.1). $\square$

Proof of Theorem 2.1, sufficiency. Assume (2.4). If $p = 0$, the single rounded subtraction $x_1 - x_1$ returns zero for every error and is an accurate evaluator. Suppose $p \neq 0$.

First determine the sign of each x_i by exactly comparing x_i with $-x_i$. Assign sign +1 when $x_i = 0$. Form the absolute values by copying or exact negation, and sort them by exact comparisons; break ties by a fixed index rule. For fixed n , this can be implemented by a finite comparison tree. No arithmetic constant is used: the signs in this description specify copying or negation, not multiplication by a stored number 1 or -1 .

This selects $T = T_{\pi,s}$ with $x = Ty$ and $y \geq 0$ as in (2.2). Compute the first $n - 1$ gaps by rounded subtraction of the exact signed raw inputs. Keep the smallest absolute input as the last gap. Hence the available values are

$$\hat{y}_i = y_i(1 + \xi_i) \quad (i < n), \quad \hat{y}_n = y_n,$$

where each ξ_i is its own rounded-node error. In particular, zero gaps are exactly zero.

On this branch, apply Lemma 6.1 to q_T , treating the last input error as zero. For a finite branch-dependent integer L_T ,

$$|P(x, \delta) - p(x)| = |\hat{q}_T - q_T(y)| \leq ((1 + u)^{L_T} - 1) W_T(y) \leq C_T((1 + u)^{L_T} - 1) |p(x)|.$$

There are only finitely many branches. Set

$$C_* = \max_T C_T, \quad L_* = \max_T L_T.$$

For every $\eta \in (0, 1)$, there is $u \in (0, 1)$ such that $C_*((1 + u)^{L_*} - 1) \leq \eta$, by continuity at $u = 0$. If $L_* = 0$, the bound is already zero. This proves (1.2) uniformly in x and all allowed errors.

At $\delta = 0$, the gap computations and subsequent polynomial computations are exact, so $P(x, 0) = p(x)$. At any zero of p , the same bound forces the output to be zero for every allowed error. Equivalently, $W_T(y) = 0$ there, so every contributing monomial has a zero gap factor and remains zero under the prescribed arithmetic.

The construction, including the branch tests and the finite arithmetic on each branch, depends only on p . It does not depend on η , u , or the values of the constants C_T . Thus it is one fixed tree valid for every requested accuracy, as required. $\square$

7 The halting decision procedure

We invoke the standard effective form of real quantifier elimination: there is a Turing algorithm that decides every first-order sentence over $\mathbb{R}$ whose atomic formulas are polynomial equalities and inequalities with rational coefficients. For a primary reference giving a complete formally verified quantifier-elimination algorithm, see [4]. That verification concerns the external quantifier-elimination algorithm, not the new coupling argument in this manuscript.

Corollary 7.1 (Proposed affirmative answer to AA-01). *There is an always-halting Turing algorithm which, from the integer coefficient list of p with $p(0) = 0$, decides whether an accurate finite tree exists in the stated model. On a positive instance it can also output the finite evaluator constructed in Section 6.*

Proof. Use the following algorithm.

1. If $p = 0$, return YES and, when requested, the tree that computes $x_1 - x_1$.
2. Enumerate the finite set of pairs $(\pi, s) \in \mathfrak{S}_n \times \{-1, 1\}^n$. For each pair, symbolically substitute (2.1) into p , expand and collect its integer coefficients, and construct q_T, W_T from (2.3).
3. For each pair decide the closed first-order sentence

$$\exists C \left(C > 0 \wedge \forall y_1 \cdots \forall y_n \left[\left(\bigwedge_{i=1}^n y_i \geq 0 \right) \implies W_T(y)^2 \leq C^2 q_T(y)^2 \right] \right). \quad (7.1)$$

If any sentence is false, return NO.

4. Otherwise return YES and construct the sign/order tree and its branch evaluators as in Section 6.

All symbolic operations are finite operations on integer coefficient lists. Sentence (7.1) involves only integer-coefficient polynomials in $C, y_1, \dots, y_n$, so its decision terminates. There are only $2^n n!$ such sentences. The whole algorithm therefore terminates.

For $y \geq 0$ we have $W_T(y) \geq 0$. Since $C > 0$, the squared inequality in (7.1) is equivalent to (2.4); this includes points where $q_T(y) = 0$. Correctness follows in both directions from Theorem 2.1.

Neither the decision procedure nor the evaluator construction requires knowing the size of a putative accurate tree. Also, the quantified constant C is used only in the symbolic decision test and the analysis; it is not a numerical constant input to the evaluator. $\square$

8 Boundary cases, examples, and scope

Example 8.1 (An allowable input difference). For $p(x) = x_i - x_j$, each q_T is either zero or, up to an overall sign, a linear form with nonnegative coefficients. Thus $W_T(y) = |q_T(y)|$ on $y \geq 0$ and the criterion accepts p . The single rounded subtraction is already an accurate evaluator.

Example 8.2 (Three-input summation). Take $p = x_1 + x_2 + x_3$ and the signed-order substitution

$$x_1 = y_1 + y_2 + y_3, \quad x_2 = -(y_2 + y_3), \quad x_3 = -y_3.$$

Then $q_T = y_1 - y_3$ and $W_T = y_1 + y_3$. At $y = (1, 1, 1)$, the former vanishes while the latter is 2. Sentence (7.1) is therefore false for this region, so the criterion rejects the polynomial. This agrees with the impossibility example discussed in [2, 3].

Example 8.3 (Bounded cancellation is permitted). Let $p(x) = x^5 - x^3 + x$. In either one-variable sign region,

$$W(y) = y^5 + y^3 + y, \quad |q(y)| = y(y^4 - y^2 + 1), \quad y \geq 0.$$

The factor $y^4 - y^2 + 1$ is strictly positive, because it equals $(y^2 - 1/2)^2 + 3/4$. Moreover,

$$3|q(y)| - W(y) = 2y(y^2 - 1)^2 \geq 0.$$

Thus $C = 3$ works, even though the polynomial has a negative coefficient. The fractions in the positivity identity belong only to this proof, not to the evaluator's constant inputs.

Closed regions and zeros. Necessity is first proved on strictly ordered, nonzero inputs, then extended by continuity to the entire nonnegative orthant of gap coordinates. Sufficiency uses those closed-region inequalities, so it covers tied magnitudes, zero coordinates, the origin, and zeros of p . No omission of a lower-dimensional set is made.

Stored values and error-dependent comparisons. The necessity proof couples actual register values and explicitly checks every provenance type of comparison. In particular, it does not turn a stored-value program into a formula with newly independent errors at reused occurrences. The formal expansion used for the deliberately constructed evaluator in Lemma 6.1 retains shared error variables and their multiplicities.

Limits of the stated result. The theorem concerns whole-space polynomial evaluation in the exact model of Section 1. It does not assert a characterization for arbitrary restricted domains, additional arithmetic primitives, nonzero constant inputs, numerical access to comparison outcomes, deterministic rounding rules tying errors at repeated operands, or bit-level floating-point algorithms. Those changes alter premises of the coupling argument. No complexity or practical-efficiency bound is claimed.

Logical dependencies. The substantive new step is Lemma 4.2. It implies Proposition 5.1; polynomial interpolation then proves necessity. A direct termwise construction proves sufficiency. The standard real-quantifier-elimination theorem yields the final Turing decision procedure. The manuscript supplies all these model-specific steps but remains a proposed proof pending human peer review or formal verification.

References

- [1] A. Townsend, *Open Problems in Numerical Linear Algebra*, AA-01: “Deciding accurate evaluability of real polynomials,” statement in the MColbrook repository used in the accompanying conversation. <https://github.com/MColbrook/OpenProblemsInNLA/blob/main/arithmetic-and-complexity/AA-01/README.md>.
- [2] J. Demmel, I. Dumitriu, and O. Holtz, *Toward accurate polynomial evaluation in rounded arithmetic*, in *Foundations of Computational Mathematics: Santander 2005*, Cambridge University Press, 2006, pp. 36–105. ArXiv version 2, 18 January 2006: <https://arxiv.org/abs/math/0508350v2>.
- [3] J. Demmel, I. Dumitriu, O. Holtz, and P. Koev, *Accurate and efficient expression evaluation and linear algebra*, *Acta Numerica* **17** (2008), 87–145. Definitions 3.3–3.4 and Section 3.3, especially Section 3.3.7. <https://doi.org/10.1017/S0962492906350015>.
- [4] K. Kosaian, Y. K. Tan, and A. Platzer, *A First Complete Algorithm for Real Quantifier Elimination in Isabelle/HOL*, CPP 2023. <https://doi.org/10.1145/3573105.3575672>. Preprint: <https://arxiv.org/abs/2209.10978v2>.